Universidad del Valle de Guatemala

Construcción de Compiladores – Sección 10

COMPISCRIPT MIPS COMPILER IDE

Informe técnico

Proyecto 3: generación de código ensamblador

Grupo: Daniel-Barillas
Integrante: Pablo Daniel Barillas Moreno
Carné: 22193
Versión: 3.0.0
Fecha: 20 de octubre de 2026

Compiscript → TAC → MIPS32 → MARS 4.5

Índice general

1. Resumen ejecutivo **2**

2. Fundamentos teóricos **3**
2.1. Fases de compilación . . . 3
2.2. Código de tres direcciones . . . 3
2.3. Arquitectura MIPS32 y MARS . . . 3
2.4. Registros de activación . . . 4

3. Diseño e implementación **5**
3.1. Contrato del backend . . . 5
3.2. Resolución de almacenamiento . . . 5
3.3. Asignador getReg . . . 5
3.4. Selección de instrucciones . . . 5
3.5. Datos compuestos . . . 6
3.6. Strings y flotantes . . . 6
3.7. Mirilla . . . 6

4. Interfaz y experiencia de uso **7**

5. CLI y ejecución externa **8**

6. Pruebas y resultados **9**
6.1. Suite automatizada . . . 9
6.2. Validación real en MARS . . . 9
6.3. Comandos de verificación . . . 9

7. Trazabilidad con el enunciado **11**

8. Limitaciones y trabajo futuro **12**

9. Conclusiones **13**

A. Mapa de archivos **14**

Capítulo 1

Resumen ejecutivo

El Proyecto 3 completa un compilador educativo para Compiscript. Conserva el lexer y parser generados con ANTLR4TS, el análisis semántico con tabla de símbolos y la representación intermedia de tres direcciones de las entregas anteriores. Sobre esa base incorpora un backend que transforma exclusivamente TAC en ensamblador MIPS32 compatible con MARS 4.5.

El producto se presenta como una interfaz gráfica React con editor Monaco. El usuario puede escribir o seleccionar un archivo `.cps`, ejecutar el pipeline, observar diagnósticos y examinar tokens, CST, árbol semántico, símbolos, ámbitos, TAC, bloques básicos, registros de activación y ensamblador. La exportación MIPS se habilita solamente cuando todas las fases terminan correctamente.

El backend implementa convención de llamadas mediante pila, asignación local de registros, aritmética entera y flotante, control de flujo, funciones, llamadas anidadas, recursión, arreglos, objetos, métodos, strings y closures. La salida fue ensamblada y ejecutada en el MARS 4.5 oficial con quince programas positivos.

Capítulo 2

Fundamentos teóricos

2.1. Fases de compilación

Un compilador transforma un lenguaje fuente en una representación destino mediante fases con contratos definidos. En este proyecto, los tokens del lexer alimentan al parser; el CST válido alimenta al visitor semántico; el resultado tipado habilita TAC; finalmente, TAC habilita MIPS.

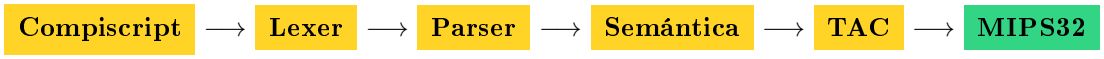


Figura 2.1: Pipeline completo de la versión 3.0.0.

El principio de bloqueo evita que un error temprano se convierta en muchos errores derivados: lexer y parser continúan recolectando diagnósticos, pero TAC y MIPS no se ejecutan si el programa no posee significado válido.

2.2. Código de tres direcciones

TAC descompone expresiones complejas en instrucciones con a lo sumo dos operandos fuente y un resultado. Por ejemplo, `x = a + b * 4` se convierte en:

```
t0 = b * 4
t1 = a + t0
x = t1
```

Esta forma simplifica la selección de instrucciones: cada cuádruplo tiene una traducción local y los metadatos de tipo determinan si debe usarse la ALU entera o el coprocesador flotante.

2.3. Arquitectura MIPS32 y MARS

MIPS emplea un conjunto reducido de instrucciones y registros explícitos. El proyecto usa registros temporales `$t0-$t9`, argumentos `$a0-$a1`, retornos `$v0`/`$f0`, stack pointer `$sp`, frame pointer `$fp`, return address `$ra` y el registro `$s7` como enlace al entorno de un closure.

MARS 4.5 ensambla MIPS32 y ofrece syscalls educativas. Se emplean syscall 1 para enteros, 2 para flotantes, 4 para strings, 9 para memoria dinámica y 10 para terminar el programa.

2.4. Registros de activación

Cada llamada necesita un frame independiente para soportar recursión. El tamaño proviene de `ActivationRecord.totalBytes` y se alinea a ocho bytes.

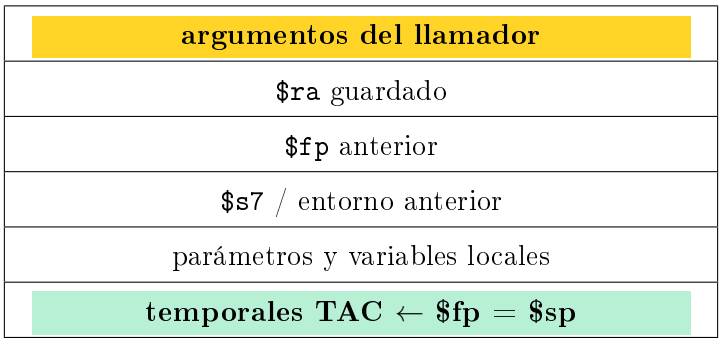


Figura 2.2: Organización conceptual del frame.

El llamador apila argumentos de izquierda a derecha. La función reserva el frame, guarda estado y copia parámetros a offsets estables. Una `CALL` consume solo sus propios argumentos, lo cual resuelve correctamente llamadas anidadas.

Capítulo 3

Diseño e implementación

3.1. Contrato del backend

`src/mips/generator.ts` expone `generateMips(tac)`. El resultado usa estados explícitos: `not-requested`, `skipped`, `completed` y `failed`. Un ensamblador parcial nunca se presenta como listo.

Estado	Significado
not-requested	la fase no fue solicitada
skipped	no existe TAC válido por errores anteriores
completed	ensamblador generado y validado
failed	el backend detectó una inconsistencia

3.2. Resolución de almacenamiento

Los operandos globales se convierten en etiquetas del segmento `.data`. Los operandos de la función activa usan su offset relativo a `$fp`. Las constantes se materializan con `li`, `la` o `l.s`. Los valores persistentes se almacenan después de cada cuádruplo; así no dependen de que un registro temporal sobreviva a saltos o llamadas.

3.3. Asignador getReg

El asignador administra `$t0` a `$t7` y reinicia su conjunto de uso en cada instrucción TAC. Esta decisión corresponde a la naturaleza de tres direcciones: los valores vivos entre instrucciones están en slots, mientras que los registros sirven para cargar operandos y producir un resultado local.

3.4. Selección de instrucciones

Familia TAC	MIPS	Observación
MOV	li/la/lw y sw	depende de la clase del operando
ADD/SUB/MUL	addu/subu/mul	enteros de 32 bits
DIV/MOD	div con mflo/mfhi	cociente o residuo
EQ–GE	seq, sne, slt, sle, sgt, sge	resultado booleano 0/1
IF/GOTO	beq, bne, j	delay slot explícito

Familia TAC	MIPS	Observación
CALL	jal o jalr	nombre directo o puntero
RETURN	$v0/$f0	salta a un epílogo común
PRINT	syscall	seleccionado por tipo

3.5. Datos compuestos

Los arreglos guardan longitud en el primer word y elementos desde el segundo. Los objetos reservan `ClassLayout.instanceSize`; los campos usan offsets calculados en TAC. Los métodos se convierten en direcciones de etiqueta. Un closure reserva un puntero a función seguido de sus capturas y usa `$s7` como entorno activo.

3.6. Strings y flotantes

Las cadenas constantes se internan para evitar duplicados. La concatenación solicita memoria y copia bytes; la igualdad compara byte a byte. Los flotantes usan instrucciones `l.s`, `s.s`, `add.s`, `sub.s`, `mul.s` y `div.s`. La promoción entera se realiza con `mtc1` y `cvt.s.w`.

3.7. Mirilla

La optimización local elimina movimientos de un registro hacia sí mismo, ajustes nulos de pila, saltos hacia la etiqueta siguiente y `nop` consecutivos. Se conserva la salida previa para auditoría y se informa cuántas instrucciones se removieron.

Capítulo 4

Interfaz y experiencia de uso

El flujo normal para Entrada válida y Archivo propio solicita modo `mips`; por tanto, un programa correcto llega automáticamente al backend. El modo TAC permanece disponible como demostración aislada.

El pipeline visual usa el estado real de cada fase. MIPS aparece como omitido ante errores previos, fallido ante diagnósticos del backend y listo para MARS solo en `completed`. El panel muestra estado, instrucciones finales, optimizaciones, diagnósticos, cantidad TAC y pico de registros.

El selector de archivos está dentro de la GUI y acepta `.cps`. Los resultados se muestran en la misma interfaz; la consola no es necesaria para la evaluación funcional.

Capítulo 5

CLI y ejecución externa

La CLI comparte `analyzeInput`; no posee un compilador alterno. El modo MIPS acepta `-out`:

```
npm run cli -- examples/mips/07_recursion.cps \
  --mode mips --out factorial.asm
java -jar Mars4_5.jar nc ae1 factorial.asm
```

El script `scripts/run-mars.mjs` busca el JAR en el segundo argumento, en `MARS_JAR` o en `tools/Mars4\_5.jar`. MARS sigue siendo una herramienta externa: el IDE genera y exporta, pero no simula internamente.

Capítulo 6

Pruebas y resultados

6.1. Suite automatizada

La suite final contiene 221 casos en 15 archivos: 186 se ejecutan sin herramientas externas y 35 validan la salida real en MARS cuando Java y el JAR están disponibles. Las pruebas del backend verifican generación completa, convención de pila, arreglos, runtime de strings, llamadas anidadas, closures, recuperación de `try/catch`, bloqueo por errores y mirilla. Las pruebas heredadas continúan verificando lexer, parser, semántica, símbolos, TAC, CFG, UI y la exportación del modo MIPS.

6.2. Validación real en MARS

Se ejecutaron quince programas positivos en MARS 4.5. Algunos resultados representativos son:

Caso	Salida
aritmética	14
funciones y pila	42
arreglos y foreach	6
strings y booleanos	Hola MIPS; true
flotantes	4.0
factorial recursivo	120
closure	1
clase, método y llamada anidada	2; 6; 12

El caso de errores mezclados produce diagnósticos, deja TAC/MIPS en `skipped` y no crea un archivo ensamblador.

6.3. Comandos de verificación

```
npm ci
npm run check
npm test
npm run build
npm run cli:mips
npm audit --omit=dev
```

TypeScript, pruebas y build terminan correctamente; la auditoría de dependencias de producción reporta cero vulnerabilidades.

Capítulo 7

Trazabilidad con el enunciado

Requisito	Estado	Evidencia
ANTLR obligatorio	Cumple	gramática y código generado
MIPS desde TAC	Cumple	API `generateMips(tac)`
GUI y selector de archivo	Cumple	React/Monaco y menú Archivo
Resultados dentro de GUI	Cumple	paneles de fases, TAC y MIPS
Múltiples errores y recuperación	Cumple	listeners, estrategia ANTLR y visitors
No TAC/MIPS ante errores	Cumple	estados y pruebas negativas
Mirilla	Cumple	optimizador y métricas
Ensamblador ejecutable	Cumple	quince ejecuciones MARS
No código objeto ni ejecución interna	Cumple	exportación a herramienta externa

Capítulo 8

Limitaciones y trabajo futuro

Compiscript y el TAC actual no definen una instrucción `throw` ni un ABI de excepciones; por ello los marcadores `TRY_BEGIN`/`CATCH_BEGIN` preservan estructura, pero no realizan unwinding de pila. Tampoco existe una fase global de optimización, de acuerdo con el alcance: solo se aplica mirilla local.

Como evolución podrían incorporarse bounds checks con handlers, asignación global de registros por liveness y un runner de integración continua que descargue MARS bajo aceptación de su licencia. Estas mejoras no son necesarias para el alcance actual.

Capítulo 9

Conclusiones

El Proyecto 3 integra de forma coherente las fases anteriores y produce ensamblador MIPS32 real desde TAC. La convención de llamadas permite funciones, anidamiento y recursión; el modelo de heap cubre arreglos, objetos, cadenas y closures; los estados impiden exportar resultados inválidos; y la interfaz hace observable todo el proceso.

La evidencia combina pruebas unitarias, integración, build de producción y ejecución en MARS 4.5. Por ello, la afirmación de compatibilidad no depende solamente de documentación o de ejemplos de ensamblador, sino de resultados reproducibles.

Apéndice A

Mapa de archivos

Archivo	Responsabilidad
`src/mips/generator.ts`	backend, runtime, mirilla y diagnósticos
`src/tac/types.ts`	contrato de TAC, frames y layouts
`src/lib/analyze.ts`	orquestación y bloqueo entre fases
`src/ui/App.tsx`	flujo completo para entrada propia
`src/ui/components/RightDock.tsx`	inspección y descarga MIPS
`src/cli/run.ts`	modo MIPS y escritura `.asm`
`scripts/run-mars.mjs`	invocación externa de MARS
`src/__tests__/mips.test.ts`	regresiones del backend
`examples/mips`	demostraciones y caso de bloqueo